



UITs
UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

Lab 6

Department: Computer Science and Engineering(CSE)

Course title: Operating System Lab

Course Code: CSE-412

Submitted To

Name: Afroja Smrity Ahmed

Lecturer, University of Information Technology & Sciences (UITs)

B.Sc in CSE, Jahangirnagar University(JU)

Submitted by

Name: Md. Shoyaif Rahman

ID: 0432310005101050

Section: 7A2

Batch: 53(Spring)

Submission date: 24/04/26

Q1. (Understanding – CLO1):

Explain the concept of Round Robin CPU scheduling. How does it differ from FCFS (First Come First Serve) scheduling in terms of process execution and fairness? Provide a suitable real-life analogy.

Answer:

Round Robin (RR):

- Each process gets a fixed time quantum (q).
- CPU cycles through processes → preemptive scheduling.
- Ensures fairness (no process waits too long).

FCFS (First Come First Serve):

- Processes execute in arrival order.
- Non-preemptive → once started, runs till completion.
- Can cause long waiting time (convoy effect).

Difference:

- RR = fair, responsive, time-sharing
- FCFS = simple but can be unfair

Real-life analogy:

- RR: Like students taking turns answering in class (everyone gets a chance).
- FCFS: Like a single queue at a bank (one person finishes fully before next).

Q2. (Applying – CLO2):

Write a C program to implement Round Robin scheduling for n processes with given burst times and a fixed time quantum. Assume all processes arrive at time 0. Test your program with at least 4 processes.

Answer:

Code:

```
#include <stdio.h>

int main() {
    int n = 4, tq = 2;
    int bt[4] = {5, 4, 2, 1};
    int rem_bt[4], wt[4] = {0}, tat[4] = {0};
    int t = 0, done;

    for(int i = 0; i < n; i++)
        rem_bt[i] = bt[i];

    do {
        done = 1;
        for(int i = 0; i < n; i++) {
            if(rem_bt[i] > 0) {
                done = 0;
                if(rem_bt[i] > tq) {
                    t += tq;
                    rem_bt[i] -= tq;
                } else {
                    t += rem_bt[i];
                    wt[i] = t - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
    } while(!done);

    for(int i = 0; i < n; i++)
        tat[i] = wt[i] + bt[i];
}
```

```

printf("P\tBT\tWT\tTAT\n");
for(int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);

return 0;
}

```

Output:

Output			
P	BT	WT	TAT
P1	5	7	12
P2	4	7	11
P3	2	4	6
P4	1	6	7

Q3. (Analyzing – CLO3:

Answer: Consider the following processes:

Proce ss	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	4
P3	2	2
P4	4	1

If the **time quantum = 2 ms**,

Construct the **execution sequence (Gantt chart)** using Round Robin scheduling.

| P1 | P2 | P3 | P1 | P4 | P2 | P1 |
0 2 4 6 8 9 11 12

Calculate the,

Turnaround time (**TAT**)=**CT-AT** .

Waiting time (**WT**) =**TAT-BT**

Process	BT	AT	CT	TAT	WT
P1	5	0	12	12	7
P2	4	1	11	10	6
P3	2	2	6	4	2
P4	1	4	9	5	4

Q4. (Evaluating – CLO4)

Discuss the impact of selecting a very small time quantum, a very large time quantum in Round Robin scheduling. How does it affect CPU utilization, context switching and response time?

Answer:

The performance of Round Robin (RR) scheduling depends on the time quantum (TQ), which is the fixed time given to each process.

If the time quantum is very small, the CPU switches between processes very frequently. This increases context switching overhead, reduces CPU efficiency, and lowers overall system performance. However, response time becomes very fast because each process gets CPU quickly.

If the time quantum is very large, Round Robin behaves like First Come First Serve (FCFS). In this case, processes wait longer for their turn, leading to poor response time, although context switching is reduced and CPU utilization improves.

Therefore, an optimal (medium) time quantum should be selected. It provides a balance between low context switching, good CPU utilization, fast response time, and fairness, making the system efficient for multitasking environments.

Q5. (Applying + Programming – CLO2 & CLO3)

Modify your Round Robin C program to include arrival time of processes. Assume processes may arrive at different times. Show how the scheduling behavior changes with a sample input.

Answer:

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 4;
```

```
    int at[] = {0,1,2,4};
```

```
    int bt[] = {5,4,2,1};
```

```
int rt[4];
```

```
int tq = 2;
```

```
int time = 0, remain = n;
```

```
for(int i=0;i<n;i++)
```

```
    rt[i] = bt[i];
```

```
while(remain > 0) {
```

```
    for(int i=0;i<n;i++) {
```

```
        if(at[i] <= time && rt[i] > 0) {
```

```
            if(rt[i] <= tq) {
```

```
                time += rt[i];
```

```
                rt[i] = 0;
```

```
                remain--;
```

```
                printf("P%d finished at %d\n", i+1, time);
```

```
            } else {
```

```
        rt[i] -= tq;

        time += tq;
    }

}

}

}
```

Output:

Output

```
P3 finished at 6
P4 finished at 7
P2 finished at 11
P1 finished at 12
```

Q6. (Creating – CLO5)

Design a simulation scenario for a time-sharing operating system (e.g., a university computer lab system). Explain how Round Robin scheduling improves system performance. Propose a optimized time quantum value for your scenario with proper justification.

Answer:

Simulation Scenario: A university computer lab where many students use computers for coding, browsing, and assignments. All programs share the CPU, so it works as a time-sharing system.

How Round Robin Improves Performance:

- Gives each process equal CPU time → fairness
- Fast response for users → no lag
- Prevents any process from blocking others → no starvation
- Supports smooth multitasking

Optimized Time Quantum: 20 ms

Justification:

- Too small → too many context switches (inefficient)
- Too large → behaves like FCFS (slow response)
- 20 ms balances speed and efficiency

